

Memory in Memory: 从下游失败中学习程序性记忆策略

匿名作者

(联系方式略)

2026 年 8 月

摘要

大语言模型 (LLM) 智能体日益依赖外部记忆来跨会话保存用户事实、状态和历史事件。现有记忆系统主要改进内容的表示、压缩与检索，却很少研究记忆系统如何从自身的下游错误中持续学习“以后应当如何记忆”。当一个问题被回答错误时，同一个失败信号可能来自流水线的不同阶段：目标信息可能从未被写入记忆，可能已经存在却未被检索到，也可能已经进入上下文却被回答模型错误使用。那些不区分这些根因、直接对失败进行反思或改写全局提示词的方法，容易产生错误归因、重复规则和全局回归。

我们提出 **Memory in Memory** (MiM)，一种面向大模型介导记忆系统的错误驱动程序性元记忆框架。MiM 将长期记忆系统分解为记忆构建策略与记忆访问策略，并在正常运行之外增加一个离线错误学习回路。给定一次下游失败，MiM 运行三项信息隔离的检验——回答证据充分性、当前有用记忆的访问覆盖、以及沿来源与版本链的记忆构建保真度——从而把访问遗漏与构建损失归因为非互斥、因果边界清晰的失败，而不是打上一个笼统的杂项标签。确认的失败被蒸馏为自然语言候选技能；候选技能经语义聚类、通过显式的候选一技能库亲和矩阵与既有技能建立关系、并由带写集冲突消解的受约束批量 CRUD 整合后，才以新的版本化技能库发布。运行时只检索冻结的正式技能，同时一条技能暴露轨迹记录实际注入的技能与检索边界附近的候选，使后续学习轮能够区分策略缺失、召回失败与执行无效。由此，MiM 让长期记忆系统积累的不再只是关于用户和世界的对象级事实，还包括关于自身构建与访问策略应当如何运作的程序性经验。

1 引言

外部记忆已成为长期运行的大语言模型智能体的基础组件 [2, 1]。具备持久记忆的智能体可以跨会话保留用户偏好、人物关系、历史事件与状态变化，并在未来任务中选择性地恢复相关信息 [3, 25, 24]。围绕这一目标，已涌现出丰富的设计空间：向量记忆、分层摘要、结构化事实、时序知识图谱、多类型记忆与智能体管理式记忆系统 [2, 4, 5, 6, 7, 8]。尽管底层表示不断增强，长期记忆系统仍然反复出现相似错误：重要信息未被保留，更新破坏了历史状态，语义相似但含义不同的事件被错误合并，已经存在的事实未被检索，或多跳问题只拿到部分证据便提前回答。

这些错误并不完全来自存储容量或检索器性能。对于大模型介导的记忆系统，关键决策发生在策略层：模型决定哪些信息值得写入、新信息应如何与既有记忆整合、当前问题应检索什么、是否

继续搜索、以及何时累积证据足以回答。即使底层存储能够精确持久化并返回数据，不恰当的构建与访问策略仍会造成系统性失败。

现有系统通常积累对象级记忆，即关于用户、环境与历史的事实；相比之下，它们很少积累程序性元记忆，即关于记忆系统自身运作方式的经验。例如，系统可以记录“用户已经搬到上海”，却未必能从先前的错误中学到“更新当前地址时，应保留旧地址的历史有效区间”；它可以保存多个相关事件，却未必能学到“对列表类问题，只找到一个组成部分时不应停止检索”。

一种直接的补救方案是保存失败案例，在遇到相似问题时检索历史错误；另一种方案是把所有反思不断追加到全局提示词。这两种方式都缺少关键的根因约束。错误答案只是下游表象：目标信息可能没有进入记忆、没有被检索到、或者已经可见却被误用。若系统把访问遗漏归纳为构建规则，或把回答推理错误写成检索策略，新经验不仅无法迁移，还可能损害无关任务。

因此，从错误中学习记忆策略首先要回答一个因果问题：

在原始交互、记忆状态、访问轨迹与回答上下文构成的流水线中，目标信息第一次在哪里不再被正确保留或使用？

我们提出 **Memory in Memory (MiM)**，将下游错误转化为可归因、可维护、可复用的程序性记忆策略。MiM 不替换底层记忆系统，而是在其外部增加一个控制平面。正常运行阶段，基座系统完全按照原有方式构建与访问记忆；离线学习阶段，MiM 观察原始交互、记忆版本、自然搜索轨迹与回答输出，诊断三类失败：回答时的证据误用、当前记忆状态上的访问遗漏、以及版本历史中的构建损失。

MiM 的诊断不是单次模型调用输出的互斥标签。回答充分性、访问遗漏与构建损失被建模为三个基于不相交信息视图的检验。访问检验在固定的当前快照上比较当前有用记忆与自然搜索结果，不读取原始对话或历史版本；构建检验先确认当前记忆是否完整，仅当发现缺口时才沿来源与版本关系定位最早的破坏点。同一个问题可以同时携带访问与构建问题：当前记忆可能丢失了一个事实，同时未能召回另一个仍然存在的事实。

对于确认的访问或构建失败，MiM 不把完整失败案例注入运行上下文，而是将其蒸馏为简短、可检索、自然语言的候选技能，描述某类情形何时出现、记忆智能体应采取什么操作原则。为防止逐案例更新导致策略库膨胀，MiM 先独立归纳候选，再对语义相近的候选聚类、检索相关正式技能、执行受约束的批量 CRUD 整合。只有经过全局整合与一致性验证，候选才成为运行时正式、可检索的策略。

MiM 还记录每次决策中实际检索到的正式技能与检索边界附近的候选。这条技能暴露轨迹让下一轮学习能够区分三种失败：策略库中不存在相关经验、相关经验存在但未进入上下文、以及相关经验已被使用但内容无效。技能学习由此不再是不可观察的提示词改写，而成为一个表示、召回与执行三个阶段可分别分析的闭环。

本文的核心贡献如下：

1. 我们提出错误驱动程序性元记忆这一问题设置：长期记忆系统应积累关于自身构建与访问策略的可复用经验，而不只是对象级事实。

2. 我们提出信息隔离的失败归因方法，将回答不足、访问遗漏与构建损失分解为非互斥检验，并利用来源与版本关系定位信息保真度首次被破坏的状态转移。
3. 我们提出访问与构建方向分离的双技能库，以及从实例级候选到正式冻结策略的两阶段晋升机制。
4. 我们提出基于候选—技能库关系、带确定性验证器的批量 CRUD 与写集冲突消解的受约束技能库演化方法，以控制重复、顺序依赖与版本振荡。
5. 我们构建一个完全可观察的长期记忆研究基座，并定义会话级冻结评测协议，分别测量最终任务质量、错误类型变化与技能的自然调用。

本文其余部分组织如下。第 2 节讨论智能体记忆、自演化记忆策略与错误驱动智能体改进的相关工作。第 3 节形式化大模型介导的记忆系统、对象级与程序性记忆以及失败信号。第 4 节详细阐述 MiM 框架，包括可追踪记忆状态、闭环访问、隔离失败归因、候选归纳、语义整合与受约束的技能库演化。第 5 节总结全文。

2 相关工作

2.1 LLM 智能体的记忆架构

持久记忆沿多条轴线展开研究。**分层记忆**按访问频率与生命周期组织信息：MemGPT (Letta) [2] 借用操作系统的工作记忆—长期存储隐喻，将记忆操作暴露为函数调用，使访问决策成为智能体工具调用循环的一部分。**结构化与联想记忆**以知识图谱或联想网络表示事实：Mem0 [5]、A-MEM [4] 与 Zep/Graphiti [8] 构建支持时间感知更新的时序知识图谱。**多类型记忆**区分情景、语义、程序等类别：MemoryOS [6] 分层短时、中时与长时存储；MIRIX [7] 通过元管理者将输入路由给六个专职记忆管理者，并配以一个按问题选择记忆组件与检索方法的访问智能体。**高效记忆**面向 token 经济：Light-Mem [9] 与 SimpleMem [10] 将交互历史压缩为易检索单元；Hindsight [11] 以 retain/recall/reflect 操作维护四类网络。近期综述 [1] 将智能体记忆形式化为 write-manage-read 循环，这一视角把策略控制——写什么、如何管理、如何读——定位为设计决策。

这些系统改进了存储什么与如何检索，但支配构建与访问的策略在部署后基本固定。MiM 在这一线工作之外做出补充：它把构建与访问策略本身当作可学习资源，不提出新的存储表示，而是为任意底层记忆架构挂接一个积累程序性经验的错误学习回路。

2.2 技能引导与自演化的记忆策略

最近一条工作线让记忆策略变得可学习。**MemSkill** [12] 将记忆构建操作重构为少量可复用、可演化的技能：控制器选择相关技能，执行器按技能生成记忆，设计师定期审查困难样例以细化或创建技能。MemSkill 是构建侧最近的邻居，但其技能选择用强化学习训练，且不学习多工具、可停止的访问策略。**EvolveMem** [13] 把完整检索配置（打分函数、融合权重、预算）暴露为动作空间，由 LLM 诊断模块在“回退—回归”保护下演化。它面向连续配置优化，而非离散、可检索、自然语

言的策略。**MemMA** [14] 在统一存储接口之外放置元思考者与查询推理者：按问题诊断证据缺口、生成正交查询、并直接修补当前实例的事实条目。**MemMA** 不积累跨实例的程序性技能；它的控制器改进当前问题，而非智能体未来的构建与访问行为。**PlugMem** [15] 是面向记忆访问的与任务无关的插件控制器，选择记忆类型并规划检索，但它是可替换模块而非从失败中学到的经验库。访问侧方面，**MRAgent** [16] 与 **MemMachine** [17] 构建了图或情景存储之上的强多工具检索智能体，但其策略在设计中即已冻结。

MiM 与这一线的区别有两点。第一，它是错误驱动、归因优先的：除非下游失败已被隔离到访问或构建阶段，否则不学习任何东西，修复目标具有因果边界。第二，它的技能是程序性元记忆：经受约束批量 CRUD 整合、版本化、可检索的自然语言策略，而非连续配置增量或逐实例事实修补。

2.3 错误驱动的智能体改进与反思

反思类方法让智能体从自身错误中改进。**Reflexion** [18] 把失败轨迹转化为存入情景缓冲的言语反馈；**Self-Refine** [19] 迭代修正输出；**Generative Agents** [3] 维护接近因与重要性索引的情景缓冲；**MemoryBank** [20] 应用遗忘曲线与周期性整合来修剪过期条目。提示词级优化，如 **OPRO** [21] 与 **LangMem** 自带的提示词优化器 [22]，根据轨迹反馈全局改写智能体的策略提示词。

这些方法有一个 MiM 直指的共同弱点：反思不区分流水线首先在哪里失败。由访问失败触发的全局提示词改写可能修改构建行为；对回答推理错误的反思可能产出检索规则。MiM 则先把每次失败归因到回答、访问或构建阶段，再归纳一个运行时按条件检索的方向性技能，使新经验作用在其根因所在的位置，且不污染无关任务。

2.4 智能体技能库

记忆之外，智能体越来越注重复用程序性技能。**Voyager** [23] 通过环境反馈维护不断增长的、可执行的代码技能库；近期智能体框架把技能当作版本化、可检索的行为单元。MiM 与“学习可复用程序而非事实”的精神一致，但基底不同：其技能是简短的、自然语言的记忆构建与访问操作原则，由离线失败归因归纳而来，并通过确定性校验、无冲突的批量演化过程维护。这使技能质量可检查、技能库可版本化——这两点是“一个不损害其所指导的对象级存储可靠性的记忆学习系统”的前提。

3 问题形式化

3.1 大模型介导的记忆系统

我们考虑一个构建与访问决策至少部分由大模型作出的长期记忆系统：

$$\mathcal{M} = (\mathcal{D}, \pi_{\text{con}}, \pi_{\text{acc}}, f_{\text{ans}}), \quad (1)$$

其中 $\mathcal{D}$ 是持久记忆状态, π_{con} 是记忆构建策略, π_{acc} 是记忆访问策略, f_{ans} 是根据问题与可见证据产生答案的任务模型。

给定第 t 个交互片段 x_t 与先前记忆状态 D_{t-1} , 构建策略产生新状态与构建轨迹:

$$D_t, \tau_t^{\text{con}} = \pi_{\text{con}}(x_t, D_{t-1}; S_t^{\text{con}}), \quad (2)$$

其中 S_t^{con} 是本轮可用的构建技能集合, τ_t^{con} 是构建轨迹 (候选提议、决策与状态转移的序列)。

给定问题 q , 访问策略通过动作序列 $a_1, \dots, a_J$ 与记忆交互:

$$a_j \sim \pi_{\text{acc}}(a_j \mid q, D_t, S_q^{\text{acc}}, h_{<j}), \quad (3)$$

其中 S_q^{acc} 是为 q 检索到的访问技能, $h_{<j}$ 包含此前的查询、工具观察与中间判断。当策略执行停止动作时, 回答模型根据累积的可见证据 M_{visible} 产生预测:

$$\hat{y} = f_{\text{ans}}(q, M_{\text{visible}}). \quad (4)$$

在当前实现中, 访问与回答由同一个闭环智能体完成, 因此停止决策与最终回答共享完整的自然搜索上下文。这一选择在第 4.10 节重新讨论。

3.2 对象级记忆与程序性记忆

对象级记忆 D_t 保存用户与世界的事实。MiM 额外维护一个独立资源: 两类程序性策略库

$$\mathcal{K} = (\mathcal{K}_{\text{con}}, \mathcal{K}_{\text{acc}}), \quad (5)$$

其中 $\mathcal{K}_{\text{con}}$ 在构建期间指导信息选择、抽取、更新、合并与历史保留; $\mathcal{K}_{\text{acc}}$ 在访问期间指导查询规划、检索路线、过滤、证据聚合与停止。

每条技能采用最小自然语言表示

$$s = (n, d, c), \quad (6)$$

其中 n 是技能名称, d 是检索描述 (“该技能何时适用”), c 是执行内容 (“应遵循什么原则”)。技能不保留训练案例中的用户事实; 它们描述可跨实体与会话迁移的处理原则。

3.3 失败信号

给定参考答案 y^* 与模型预测 $\hat{y}$, 语义裁判输出

$$J(\hat{y}, y^*) \in \{\text{C}, \text{P}, \text{I}\}, \quad (7)$$

其中 C、P、I 分别表示正确、部分正确与错误。只有 P 与 I 进入失败归因。裁判只观察问题、参考答案、预测与对话时间锚点, 不读取记忆状态或搜索轨迹, 因此它只确认下游失败而不解释原因。

设 $\text{Cover}(M, y^*)$ 表示证据集 M 对 y^* 必要事实的覆盖程度, $\text{Support}(m, y^*)$ 表示记忆条目 m 是否支持至少一个必要事实。MiM 定义三种诊断结果。

定义 3.1 (回答失败). 当预测不正确、且回答模型可见的证据已经充分时, 发生回答失败:

$$F_{\text{ans}} = \mathbb{I}[J(\hat{y}, y^*) \neq C \wedge \text{Cover}(M_{\text{visible}}, y^*) = 1]. \quad (8)$$

定义 3.2 (访问失败). 设当前有用记忆为

$$M_{\text{useful}} = \{m \in D_t : \text{Support}(m, y^*) = 1\}, \quad (9)$$

自然搜索链返回的记忆为

$$M_{\text{retrieved}} = \bigcup_{j=1}^J M_j. \quad (10)$$

当有用记忆存在但未被检索到时, 发生访问失败:

$$F_{\text{acc}} = \mathbb{I}[M_{\text{useful}} \setminus M_{\text{retrieved}} \neq \emptyset]. \quad (11)$$

定义 3.3 (构建失败). 当原始交互支持必要事实、而当前记忆状态不再覆盖它们时, 发生构建失败:

$$F_{\text{con}} = \mathbb{I}[\text{Support}(X_{\text{raw}}, y^*) = 1 \wedge \text{Cover}(D_t, y^*) < 1], \quad (12)$$

其中 X_{raw} 表示原始来源证据。

这些结果不是互斥的。特别地, F_{acc} 与 F_{con} 可以同时成立: 当前记忆可能丢失了一个事实 (构建), 同时未能召回另一个仍然存在的事实 (访问)。因此 MiM 从不把失败压缩为单一标签: 三个检验在不相交的信息视图上独立计算 (见第 4.4 节)。

4 方法: Memory in Memory

4.1 总览

MiM 在基座记忆系统之外增加错误学习控制平面。一次完整迭代包含三个阶段:

1. 基座系统在冻结的技能库下构建记忆并回答训练问题;
2. MiM 对语义失败执行信息隔离的根因诊断并归纳候选技能;
3. 候选经全局整合成为新的技能库版本, 在验证数据上被选中后冻结, 供后续运行使用。

这一设计把通常被混为一谈的三件事分开。诊断回答“这次失败需要学习什么”; 整合回答“这一修复与既有策略是什么关系”; 冻结评测回答“更新后的技能库在未见会话上是否有效”。算法 1 概括了训练期循环。

Algorithm 1 失败驱动的技能学习（训练阶段）**Require:** 会话、问题、训练划分、冻结技能库 $\mathcal{K}^{(v)}$

-
- 1: 阶段 1——运行。
 - 2: **for** 训练划分中的每个会话 c **do**
 - 3: $D_t \leftarrow \text{INGEST}(c; \mathcal{K}_{\text{con}}^{(v)})$ ▷ 批量 CRUD, 单事务
 - 4: **for** 会话 c 中的每个问题 q **do**
 - 5: $(\hat{y}, M_{\text{visible}}, \tau^{\text{acc}}) \leftarrow \text{ASK}(q; \mathcal{K}_{\text{acc}}^{(v)})$
 - 6: 记录技能暴露轨迹 $T_q = (\mathcal{S}_q^{\text{selected}}, \mathcal{S}_q^{\text{nearby}}, v)$
 - 7: **end for**
 - 8: **end for**
 - 9: 阶段 2——归因。
 - 10: **for** $J(\hat{y}, y^*) \neq C$ 的每个失败问题 **do**
 - 11: F_{ans} : 仅在 M_{visible} 上做回答检查
 - 12: F_{acc} : 在当前快照 D_t 上比较有用 vs. 已检索
 - 13: F_{con} : 筛选 D_t ; 若发现缺口, 追踪首错 t^*
 - 14: **for** 确认失败的 side $z \in \{\text{acc}, \text{con}\}$ **do**
 - 15: $c_i \leftarrow G_{\text{cand}}(d_i, T_i)$ ▷ 或 $c_i = \emptyset$ (无需修改)
 - 16: **end for**
 - 17: **end for**
 - 18: 阶段 3——整合并发布。
 - 19: $(\mathcal{K}_{\text{acc}}^{(v+1)}, \mathcal{K}_{\text{con}}^{(v+1)}) \leftarrow \text{CONSOLIDATE}(\text{候选}, \mathcal{K}^{(v)})$ ▷ 见算法 2
 - 20: 在验证划分上选择技能库版本; 冻结用于评测
-

4.2 可追踪的记忆状态

归因构建失败要求记忆系统暴露来源与版本关系。我们将每个记忆状态表示为

$$D_t = (V_t, E_{\text{src}}, E_{\text{par}}, E_{\text{chg}}), \quad (13)$$

其中 V_t 是系统时间 t 可见的记忆版本集合, E_{src} 把版本连接到其来源原始消息, E_{par} 表示更新或合并引入的父子关系, E_{chg} 记录产生新版本的状态转移。每个转移由操作

$$o_t \in \{\text{ADD}, \text{UPDATE}, \text{MERGE}, \text{DELETE}, \text{SKIP}\} \quad (14)$$

产生, 来源关系随更新继承, 因此系统可以从任意原始消息追踪到它最初形成的候选、初始版本以及之后的每一次变化。

这一表示把构建诊断从“最终记忆是否错误”提升为“目标信息在哪一次状态转移中第一次被破坏”。设 $\mathcal{I}(D_t, X)$ 为“记忆状态 D_t 仍忠实保留来源证据 X 中的目标信息”这一不变量, 则最早错误点为

$$t^* = \min \{ t : \mathcal{I}(D_{t-1}, X) = 1 \wedge \mathcal{I}(D_t, X) = 0 \}. \quad (15)$$

如果初始抽取已经错误, 则 t^* 对应候选形成; 如果初始状态正确, t^* 对应第一次破坏该不变量的更新、合并或删除。

4.3 闭环记忆访问

MiM 不把访问建模为一次固定检索。访问策略在每次观察后选择搜索、检查或回答动作。第 j 轮, 策略接收此前历史 $h_{<j}$, 发出动作 a_j , 收到工具观察 o_j , 并更新

$$h_j = h_{j-1} \cup \{a_j, o_j\}. \quad (16)$$

搜索动作可以在语义、词法、精确匹配与结构化时序路线之间选择。对混合检索, 候选得分为

$$s(m | q) = \sum_{r \in \mathcal{R}} \frac{w_r}{k + \text{rank}_r(m)}, \quad (17)$$

其中 $\mathcal{R}$ 是权重 w_r 归一 (倒排秩融合) 的检索路线集合, 并进一步用实体匹配、目标时间有效性与当前状态活跃度因子校正。

智能体根据累积证据判断必要事实是否完整。信息不足时, 它可以分解问题、改变查询表达、切换路线或加深搜索; 信息充分时立即停止。这一闭环给访问技能提供了真实可控的对象: 技能不仅能影响最终查询文本, 还能影响检索路线、继续条件与证据完整性判断。

4.4 隔离的失败归因

MiM 为同一个失败构建三个独立信息视图。

回答视图。 回答检查只看到运行模型实际可见的证据, 判断错误是否发生在证据利用阶段 (定义 3.1)。它不产生修复包, 不进入技能学习; 它被记录下来, 使得当记忆证据已经充分时, 失败永远不会被误归因到记忆。

访问视图。 访问检查固定在当前记忆快照上, 只包含当前相关记忆与自然访问轨迹。它不读取原始对话或历史版本, 以免把上游构建错误引入访问归因。语义模型标记有用条目, 确定性程序计算差值

$$M_{\text{missing}} = M_{\text{useful}} \setminus M_{\text{retrieved}}. \quad (18)$$

构建视图。 构建检查采用渐进式开放。第一阶段只检查当前相关记忆对必要事实的覆盖(FULL/PARTIAL/MISSING)仅当发现问题时, 第二阶段才打开原始来源消息与版本轨迹, 通过式 (15) 定位 t^* 。

这一权限设计的目的不是减少模型输入, 而是建立可解释的因果边界: 访问失败意味着“可用信息没有被检索”, 构建失败意味着“当前信息状态本身已经错误”。由于三个工作流在物理与上下文上相互隔离, 它们可以并行运行并产出独立的修复目标。

4.5 运行时技能暴露

运行时从方向对应的技能库检索:

$$\mathcal{S}_q = \text{TopK}_{s \in \mathcal{K}_z} [\lambda \cdot \text{sim}_{\text{sem}}(q, s) + (1 - \lambda) \cdot \text{sim}_{\text{lex}}(q, s)], \quad z \in \{\text{acc}, \text{con}\}, \quad (19)$$

其中语义相似度在技能名称与描述(触发条件)上计算, 词法相似度为 BM25 得分。先取出固定候选池, 再由基于 LLM 的适用性重排器过滤不适用的技能, 并用量化阈值拒绝低置信匹配。本配置中 $\lambda = 0.7$, 每次决策至多注入两条技能, 并保留随后的五条边界候选。

除注入技能外, 系统还记录检索边界附近、未进入上下文的候选:

$$T_q = (\mathcal{S}_q^{\text{selected}}, \mathcal{S}_q^{\text{nearby}}, v_{\mathcal{K}}), \quad (20)$$

其中 $v_{\mathcal{K}}$ 是正式技能库版本。 T_q 是供后续学习轮消费的策略暴露轨迹。如果相关技能已进入 $\mathcal{S}_q^{\text{selected}}$ 而失败依旧, 问题很可能在技能内容或执行; 如果它只出现在 $\mathcal{S}_q^{\text{nearby}}$, 问题很可能在检索表示; 如果两者都不出现, 系统可能需要学习新策略。这三种情形——策略缺失、召回失败、执行无效——由此可分离, 而这正是提示词级反思方法无法观察到的。

4.6 失败条件下的技能归纳

对每个诊断包 d_i 及其策略暴露轨迹 T_i , 候选智能体产出

$$c_i = G_{\text{cand}}(d_i, T_i), \quad (21)$$

其中 c_i 包含名称、检索描述、执行内容与一句“它解决的一般问题”的说明。生成过程被要求移除具体实体、日期、答案与内部 ID, 以保证候选跨案例可复用。

候选智能体也可以输出

$$c_i = \emptyset, \quad (22)$$

表示既有策略已覆盖该问题, 或该问题不适合通过记忆技能修复。空操作是控制技能库增长的有意机制, 而非生成失败。

发布前, 候选不能被运行时检索。MiM 由此建立两阶段策略生命周期: 实例级失败先产生不可执行的修复假设, 再由全局整合过程决定其新增、合并、更新或拒绝。

4.7 语义候选整合

逐错误更新策略库会引入强顺序依赖：为一个错误发布的技能会改变后一个错误看到的技能库，相似失败可能创建重复规则，同一技能可能被反复改写。MiM 因此先收集候选，再批量整合。候选的语义表示为

$$e(c_i) = \alpha e_{\text{desc}}(c_i) + \beta e_{\text{content}}(c_i) + \gamma e_{\text{solves}}(c_i), \quad (23)$$

其中 $\alpha + \beta + \gamma = 1$ （本配置为 0.45/0.35/0.20）。语义相近的候选由确定性球面 K -means 聚为候选簇 $\{C_g\}_{g=1}^G$ ，受目标簇规模与最大批量规模约束。

对候选 c_i 与既有技能 s_j ，MiM 显式计算关系：

$$R_{ij} = \lambda_d \text{sim}_{\text{sem}}(c_i^d, s_j^d) + \lambda_c \text{sim}_{\text{sem}}(c_i^c, s_j^c) + \lambda_l \text{sim}_{\text{lex}}(c_i, s_j), \quad (24)$$

其中 $\lambda_d + \lambda_c + \lambda_l = 1$ （本配置为 0.50/0.30/0.20）。与把整组候选拼接成单条查询相比，矩阵 R 保留了每个修复假设自己的相关技能，同时仍允许发现覆盖多个候选的公共策略。规划前，最优关系分超过阈值（本配置 0.60）的候选被关联到对应正式技能。

4.8 受约束的技能库演化

对候选簇 C_g 、相关既有技能与关系矩阵 R_g ，CRUD 智能体产生更新计划

$$P_g = G_{\text{crud}}(C_g, \mathcal{K}_z^{(v)}, R_g). \quad (25)$$

计划可以新增技能、重命名技能、更新描述、增删改内容、移动内容或失效技能。每个候选必须获得唯一处理结果（如创建、并入既有技能、已被覆盖、拒绝）。模型只提出计划，确定性验证器检查目标存在性、方向、预期技能库版本、内容位置与候选覆盖：

$$\text{Valid}(P_g, \mathcal{K}_z^{(v)}) = 1. \quad (26)$$

只有合法计划才能修改技能库。

不同候选簇可以在同一冻结版本上并行规划。设 W_g 是计划 P_g 的写集（其修改的技能集合）。若

$$W_i \cap W_j \neq \emptyset, \quad (27)$$

两个计划冲突。MiM 构建冲突图，并对每个连通分量重新生成联合计划——合并候选集、对正式技能库重新检索后重规划。所有冲突消除后，更新以单个事务应用，产生新版本：

$$\mathcal{K}_z^{(v+1)} = \text{Apply}\left(\mathcal{K}_z^{(v)}, \bigcup_g P_g\right). \quad (28)$$

Algorithm 2 整合: 候选簇 $\rightarrow$ 新技能库版本**Require:** 候选 $\{c_i\}$, 正式技能库 $\mathcal{K}_z^{(v)}$ (冻结)

```

1: 聚类候选:  $\{\mathcal{C}_g\}_{g=1}^G \leftarrow \text{Cluster}(\{e(c_i)\})$ 
2: for 每个簇  $\mathcal{C}_g$  do
3:   检索候选—技能库关系  $R_g$ ; 关联最优关系  $\geq 0.60$  的技能
4:    $P_g \leftarrow \text{Gcrud}(\mathcal{C}_g, \mathcal{K}_z^{(v)}, R_g)$  ▷ 最多重试 3 次
5:   if  $\text{Valid}(P_g, \mathcal{K}_z^{(v)}) = 0$  then
6:     拒绝该簇
7:   end if
8: end for
9: repeat
10:  构建冲突图: 若  $W_i \cap W_j \neq \emptyset$  则  $(i, j)$  有边
11:  for 每个含  $\geq 2$  个计划的连通分量 do
12:    合并候选、重新检索、联合重规划
13:  end for
14: until 无冲突
15: 运行发布质量门 (载荷校验、候选支持度)
16:  $\mathcal{K}_z^{(v+1)} \leftarrow \text{Apply}(\mathcal{K}_z^{(v)}, \bigcup_g P_g)$  return  $\mathcal{K}_z^{(v+1)}$ 

```

这一过程把 LLM 的语义整合能力与确定性状态管理结合: 模型决定策略内容如何组织, 程序保证更新的一致性与可追踪性。算法 2 概括了该流水线。

4.9 验证与冻结评测

MiM 在训练失败上归纳候选, 在验证会话上选择技能库版本, 并冻结用于测试:

$$v^* = \arg \max_v \text{F1}_{\text{val}}(\mathcal{K}^{(v)}), \quad (29)$$

其中 F1_{val} 是验证划分上的 Token-F1, 在与测试相同的冻结协议下计算。测试时不提供参考答案、不运行诊断、绝不更新技能库。基座系统与 MiM 使用完全相同的模型、存储、工具、步骤与 token 预算, 唯一差异是冻结技能的检索与注入。为检测潜在的答案泄漏, 技能须接受具体实体、日期、内部 ID 与罕见字符串审计, 并且在会话不相交的划分 (本设置 6:2:2) 上评测, 保证人物、历史与事件不跨划分边界。

除最大化任务质量外, MiM 的长期目标还控制策略数量、冗余与更新振荡:

$$\max_{\mathcal{K}} Q(\mathcal{K}) - \alpha |\mathcal{K}| - \beta \text{Redundancy}(\mathcal{K}) - \eta \text{Churn}(\mathcal{K}) - \mu \text{Cost}(\mathcal{K}), \quad (30)$$

其中 $Q(\mathcal{K})$ 是冻结技能库下的任务质量。当前方法通过离散候选、批量 CRUD 与版本选择近似这一目标, 不依赖梯度训练。

4.10 设计论证

为什么分开构建技能与访问技能。 构建与访问作用于不同状态。构建技能改变未来记忆状态，一次错误更新可能影响后续大量问题；访问技能不修改记忆，只影响当前问题如何获取证据。它们的可观察轨迹、失败条件与验证方式也各不相同。单一共享技能库会让词汇相似但因果位置不同的规则相互干扰——例如“处理多个时间版本”既可能指构建时保留有效区间，也可能指访问时选择目标时间。方向分离的技能库保持修复责任清晰。

为什么把访问与回答合并到一个循环。 访问是否继续取决于模型对当前证据的理解。如果访问智能体先产出固定证据集再交给独立回答智能体，后者将无法动态补全缺失信息。MiM 把检索与回答放在同一 ReAct 上下文中，使模型能在每次观察后判断 FULL/PARTIAL/NONE 并自主停止。这一合并不影响归因：离线回答检查仍会检验运行模型最终可见的信息是否已经充分。

为什么只在离线使用金标准标注。 参考答案与证据仅作为离线监督，用于判断失败与定位训练阶段的修复目标。最终测试不提供任何 gold，也不允许错误驱动更新。这类似于对模型参数的有监督训练：gold 用于学习策略，但冻结后的系统必须在未见会话上自然检索并执行这些策略。

为什么批量整合候选。 候选是从单个失败归纳出的局部修复假设，不一定对应独立的长期技能：多个错误可能共享同一原因，一个错误可能已被既有技能覆盖。批量先保留实例级归纳的独立性，再施加全局压缩，从而降低样本顺序对技能库结构的影响，并为重复率、合并率与版本振荡提供显式测量对象。

5 结论

本文研究长期记忆系统如何从自身错误中学习可复用的记忆策略。MiM 将下游失败分解为回答证据利用、当前记忆访问与历史记忆构建问题，并利用来源、版本与自然搜索轨迹形成可审计的修复目标。确认的访问与构建失败被蒸馏为按方向分离（访问 / 构建）的候选技能，再通过全局关系建模、冲突约束演化与版本化发布成为正式技能库。该框架由此把长期积累的对象从“关于用户与世界的事实”扩展为“关于系统自身应如何构建与访问记忆的程序性经验”，并将策略的存在性、召回与有效性转化为可分别观察与评估的过程。

参考文献

- [1] P. Du, “Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers,” arXiv:2603.07670, 2026.
- [2] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560, 2023.

- [3] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. UIST*, 2023.
- [4] W. Zhang, J. Deng, X. Zheng, and X. Zhu, "A-MEM: Agentic memory for LLM agents," arXiv:2502.12110, 2025.
- [5] D. Chattopadhyay, A. Banerjee, D. K. Harnal, W. N. Kagitha, R. Prasad, S. Singh, A. Sharma, P. Duttagupta, and M. Lohia, "Mem0: Building production-ready AI memories for LLM applications," arXiv:2504.19413, 2025.
- [6] Z. Chen, X. Liu, X. Li, F. Zhou, C. Fu, and L. Li, "MemoryOS: A multimodal memory system for AI applications," arXiv:2506.06326, 2025.
- [7] T. Vlašić and T. Horvat, "MIRIX: A multi-agent system architecture for long-term memory," arXiv:2507.07957, 2025.
- [8] Z. Zhu, R. R. Martinez, C. Zhou, Z. Li, and X. S. Lin, "Graphiti: Incremental temporal knowledge graph construction for evolving world state," arXiv:2501.13956, 2025.
- [9] Z. Luo, J. He, S. Dai, R. Ma, B. Wang, and Y. Deng, "LightMem: A lightweight memory system for long-context LLM agents," in *Proc. ICLR*, 2026.
- [10] J. Liu, Y. Su, P. Xia, S. Han, Z. Zheng, C. Xie, M. Ding, and H. Yao, "SimpleMem: Efficient lifelong memory for LLM agents," arXiv:2601.02553, 2026.
- [11] J. van den Brandt, F. Stolp, T. de Groot, and J. F. Santos, "Hindsight: Post-hoc memory-based learning for LLM agents," in *Proc. ACL System Demonstrations*, 2026.
- [12] H. Zhang, Q. Long, J. Bao, T. Feng, W. Zhang, H. Yue, and W. Wang, "MemSkill: Learning and evolving memory skills for self-evolving agents," arXiv:2602.02474, 2026.
- [13] J. Liu, X. Ye, P. Xia, Z. Zheng, C. Xie, M. Ding, and H. Yao, "EvolveMem: Self-evolving memory architecture via AutoResearch for LLM agents," arXiv:2605.13941, 2026.
- [14] MemMA contributors, "MemMA: A meta-thinking approach for joint coordination of memory construction and access in LLM agents," arXiv:2603.18718, 2026.
- [15] PlugMem contributors, "PlugMem: A task-agnostic plug-in memory module for LLM agents," arXiv:2603.03296, 2026.
- [16] S. Ji and colleagues, "MRAgent: Active memory reconstruction with a graph-based retrieval agent for long-term LLM agents," in *Proc. ICML*, 2026.

- [17] MemMachine contributors, “MemMachine: Ground-truth-preserving episodic memory for LLM agents,” arXiv:2604.04853, 2026.
- [18] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Proc. NeurIPS*, 2023.
- [19] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dy, S. Gupta, A. Borgida, K. Sridhar, W. Yih, and D. Roth, “Self-Refine: Iterative refinement with self-feedback,” in *Proc. NeurIPS*, 2023.
- [20] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, “MemoryBank: Enhancing large language models with long-term memory,” in *Proc. AAAI*, 2024.
- [21] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, “Large language models as optimizers,” in *Proc. ICLR*, 2024.
- [22] LangChain AI, “LangMem SDK: Long-term memory for LangGraph agents,” <https://github.com/langchain-ai/langmem>, 2025.
- [23] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *TMLR*, 2024.
- [24] A. Maharana, D. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, “Evaluating very long-term conversational memory of LLM agents,” in *Proc. ACL*, 2024.
- [25] D. Wu, C. Zhao, Y. Ma, and S. Krishnan, “LongMemEval: Benchmarking chat assistants on long-term interactive memory,” arXiv:2410.10813, 2024.